ZERO TRUST AUTHORIZATION AND POLICY ENFORCEMENT FOR SECURE IOT OTA FIRMWARE UPDATES

Jayaweera N.S

IT22362780

The dissertation was submitted in partial fulfilment of the requirements for the B.Sc. (Honors) degree in Information Technology

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology

April 2026

Declaration

I declare that this is my own work, and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text. Also, I hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute my dissertation in whole or part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as article or books).

Signature: Date:

The above candidate has carried out research for the bachelor's degree dissertation under my supervision.

Signature of the Supervisor: Date:

Abstract

Over the Air (OTA) firmware update mechanisms in Internet of Things (IoT) environments are increasingly targeted by adversaries due to static trust models, lack of continuous verification, and limited auditability in existing solutions. This paper proposes a Zero Trust Authorization and Policy Enforcement Layer as a core component of a multi layered OTA security framework for resource constrained IoT devices. The proposed approach introduces an adaptive, five phase authentication mechanism that combines cryptographic identity, hardware bound enrollment, challenge response authentication, context bound proof (CBCCR), and adaptive trust scoring to enforce continuous, per transaction device validation. A dynamic trust scoring model is employed to assess device integrity, contextual consistency, and temporal behavior before authorizing updates. The framework further integrates fine grained access control and comprehensive audit logging to ensure least-privilege enforcement and full forensic traceability. Security evaluation through simulated attack scenarios demonstrates effective detection of replay attacks, device impersonation, and integrity violations. The results indicate that Zero Trust principles can be effectively applied to OTA update mechanisms in IoT systems while maintaining low computational overhead, making the approach suitable for resource constrained environments.

Keywords: Zero-Trust, IoT security, OTA firmware update, device authentication, firmware attestation, RBAC, audit logging, ECC, hardware fingerprinting

Table of Content

List of Tables

List of Figures

List of Abbreviations

Abbreviation	Description
OTA	Over the Air
IoT	Internet of things
MQTT	Message Queuing Telemetry Transport
RBAC	Role based access control
PBAC	Policy based access control
CB-CCR	Context bound challenge response
ECDSA	Elliptic Curve Digital Signature Algorithm
MCU UID	Microcontroller Unit Unique Identifier
Bootloader CRC	Bootloader Cyclic Redundancy Check
SSID	Service Set Identifier
CBC	Cipher Block Chaining

CHAPTER 1: INTRODUCTION

1.1 Background and Literature Survey

1.1.1 Background

The rapid growth of the Internet of Things (IoT) has transformed many industries such as healthcare, manufacturing, energy, and transportation. IoT devices are now widely used in critical systems, enabling real time monitoring and automation. To maintain these devices, regular firmware updates are necessary. Over the Air (OTA) updates have become the preferred method, as they allow remote updates without physical access, saving both time and cost [1]. Despite their advantages, OTA updates introduce significant security risks. Attackers can exploit insecure update channels to deliver malicious firmware, impersonate devices, or intercept communications. This can lead to device compromise, data breaches, and large-scale attacks, especially since a single vulnerability in the update process can affect many devices at once [2].

Traditional security models rely on perimeter-based trust, which is not suitable for highly distributed IoT environments. Existing frameworks like Uptane and TUF ensure firmware integrity and authenticity but follow a static trust model. Once a device is registered, it is trusted throughout its lifecycle, even if it becomes compromised. This creates security gaps, as compromised or tampered devices may still receive updates [3].

To overcome these limitations, Zero Trust security follows the principle of "never trust, always verify." In OTA systems, this means every update request must be verified before access is granted. This includes authenticating device identity, checking device integrity through attestation, and enforcing policy based access control. Additionally, continuous monitoring and logging help detect abnormal behavior and prevent compromised devices from accessing update services. By applying Zero Trust principles, OTA firmware delivery becomes more secure and resilient against modern cyber threats.

1.1.2 Literature review

Saad El Jaouhari (2022) targets insecure OTA updates on highly constrained IoT devices by proposing a SecFOTA architecture. The paper defines roles (Manufacturer, Supplier, Gateway, IoT Device) and a trust chain for firmware delivery [5]. Its approach uses public-key signatures and secure channels where the gateway verifies signed manifests and firmware images, and can perform mutual authentication with suppliers to ensure end to end integrity. It also includes an attestation step, where the IoT device sends a proof of successful update back to the gateway. While SecFOTA establishes a useful chain of trust from manufacturer to device, it does not implement dynamic access policies or comprehensive audit logging, leaving it unable to respond to runtime changes in device trust status.

N. Asokan et al. (2018) present ASSURED, a secure firmware-update framework for large-scale IoT deployments. They introduce an authorization token bound to each firmware package, where the manufacturer signs an update and includes a token encoding device specific constraint [6]. ASSURED addresses end to end authenticity effectively, but its static token model lacks per request policy flexibility and provides no mechanism for detecting changes in device hardware state between update cycles. It does not inherently provide audit logs or continuous attestation during operation.

Meha James et al. (2024) proposes a Zero-Trust architecture for general IoT networking, specifically MQTT messaging. They embed an ABAC engine inside the MQTT broker as a Policy Enforcement Point (PEP) [7]. In experiments on an IoT cyber-range, this in-broker enforcement adds only approximately 20–45 ms of latency versus over 100 ms for traditional setups. However, this work targets the communication layer rather than firmware updates, so it does not address firmware authenticity or attestation at the OTA delivery level.

Stefan Hristozov et al. (2018) present a DICE-based scheme for on-device runtime attestation for tiny IoT microcontrollers, where the device isolates an attestation routine

in privileged mode using a standard MPU and lockable ROM secret [8]. The MCU executes isolated code to measure its firmware hash and returns a signed report. Its strength is feasibility on low-cost devices without TPM or TrustZone. However, the scheme only generates attestations without addressing how updates are fetched, authorized, or logged.

Chae-Yeon Park et al. (2025) propose a lightweight FOTA scheme to thwart man-in-the-middle attacks on firmware downloads, applying server-side DEFLATE compression plus a dual-key XOR cipher on the firmware. While this addresses confidentiality and performance, the scheme does not handle device authentication, authorization, or trust management.

Chunwen Liu et al. (2024) provide a broad survey of Zero Trust in IoT contexts, analyzing how ZT principles apply to IoT's heterogeneous devices [9]. The paper identifies key ZT components including continuous device authentication, PEP/PDP logic, and asset-centric verification in an IoT setting. Its findings underscore the need for continuous authentication and policy control but stop at conceptual principles without a concrete implementation.

1.2 Research Gap

All existing secure OTA update mechanisms for IoT devices, such as Uptane and TUF, focus on ensuring firmware integrity and authenticity but not real-time, per-request device verification and adaptive access control. Current authorization approaches are static in nature, which allows updates to be delivered to any enrolled device regardless of its compliance state or security posture. While device attestation techniques exist, they are rarely integrated directly into the OTA authorization process. Furthermore, audit logging of update transactions is often minimal, reducing auditability and forensic capability.

The central research gap addressed in this work is the absence of a comprehensive, integrated Zero-Trust authorization and policy enforcement framework for OTA updates that enforces per-transaction verification, adaptive access policies, and audit trails. Specifically, no prior solution combines mandatory dynamic role and policy based access control, hardware-bound device identity, firmware attestation, quantitative trust scoring, and comprehensive audit logging into a single, operationalized, and resource-appropriate system for constrained IoT devices. Table I in the finalized research paper demonstrates that Uptane/TUF, ASSURED, and SecFOTA all rely on static trust models and lack per-transaction verification, hardware binding, dynamic policy enforcement, and quantitative trust scoring, all of which the proposed system addresses.

Table 1: Comparison with existing OTA Frameworks

Feature	SecFOTA	ASSURED	Meha James	Hristozov	SecureTrustOTA
Multi-phase auth	No	No	No	No	Yes
Trust scoring	No	No	No	No	Yes
Hardware drift detection	No	No	No	No	Yes
CB-CCR combined signing	No	No	No	No	Yes
Replay prevention	No	Partial	No	No	Yes

Audit logging	No	No	No	No	Yes
Dynamic RBAC	No	No	MQTT only	No	Yes
OTA specific	Yes	Yes	No	No	Yes

1.3 Research Problem

The existing frameworks do not support continuous device verification for each update request and lack features such as real time hardware integrity checks, dynamic access control, and proper audit logging. As a result of this, compromised devices can still receive updates without any re evaluation of their security status. This makes the OTA update process vulnerable to threats such as unauthorized firmware delivery, replay attacks, and insider attacks, while also making it difficult to track and investigate security incidents.

Therefore, there is a need for a more secure approach that can continuously verify device trust during the OTA process. This research focuses on developing a unified Zero Trust based framework that combines hardware-based device identity, dynamic access control, adaptive trust evaluation, and tamper evident audit logging, while still being suitable for resource constrained IoT devices.

1.4 Research Objectives

The main objective is to design and implement a Zero Trust Authorization and Policy Enforcement Layer for IoT OTA firmware delivery that enforces continuous, per transaction device verification through hardware bound cryptographic identity, adaptive trust scoring, dynamic role and policy based access control, and comprehensive audit logging.

Specific Objectives:

- Implement a Five phase adaptive authentication pipeline: Design and implement a progressive authentication mechanism that combines baseline cryptographic identity (ECC P-256), hardware bound enrollment via MCU fingerprinting, challenge response authentication using ECDSA, context bound proof (CB-CCR), and adaptive trust scoring on a 100 point scale.
- Develop hardware bound device identity: Bind cryptographic device identity to physical hardware characteristics (MCU UID, flash storage capacity, bootloader CRC, secure boot flag) to detect physical tampering, device replacement, and credential cloning.
- Design dynamic policy enforcement: Implement role and policy based access control (RBAC/PBAC) for OTA update distribution, enforcing the principle of least privilege and restricting firmware delivery based on device roles, compliance status, and trust score thresholds.
- Integrate firmware attestation and end to end encryption: Implement SHA-256 based firmware attestation for pre deployment integrity verification and AES-256-CBC encryption for end to end firmware protection.
- Build a comprehensive audit logging system**:** Maintain tamper evident audit logs across nine categories covering device actions, authentication events, policy enforcement decisions, hardware verification results, and trust scoring for forensic analysis and compliance reporting.

CHAPTER 2: METHODOLOGY

2.1 Overall System Architecture

The proposed component, the Zero Trust Authorization and Policy Enforcement Layer is implemented as Layer 1 of the broader SecureTrustOTA framework. The overall SecureTrustOTA framework operates through the cooperation of four tightly integrated layers. Layer 1 (Green) is the Zero Trust Authorization and Policy Enforcement Layer, which is the focus of this dissertation. Layer 2 (Blue) is the Lightweight Secure OTA Core responsible for encrypted firmware delivery using ECC and AES-CCM. Layer 3 (Orange) is the Threshold based Anomaly Detection Engine that continuously monitors telemetry data. Layer 4 (Grey) is the Resilient Fail Safe Update Engine that handles dual slot updates and rollback protection.

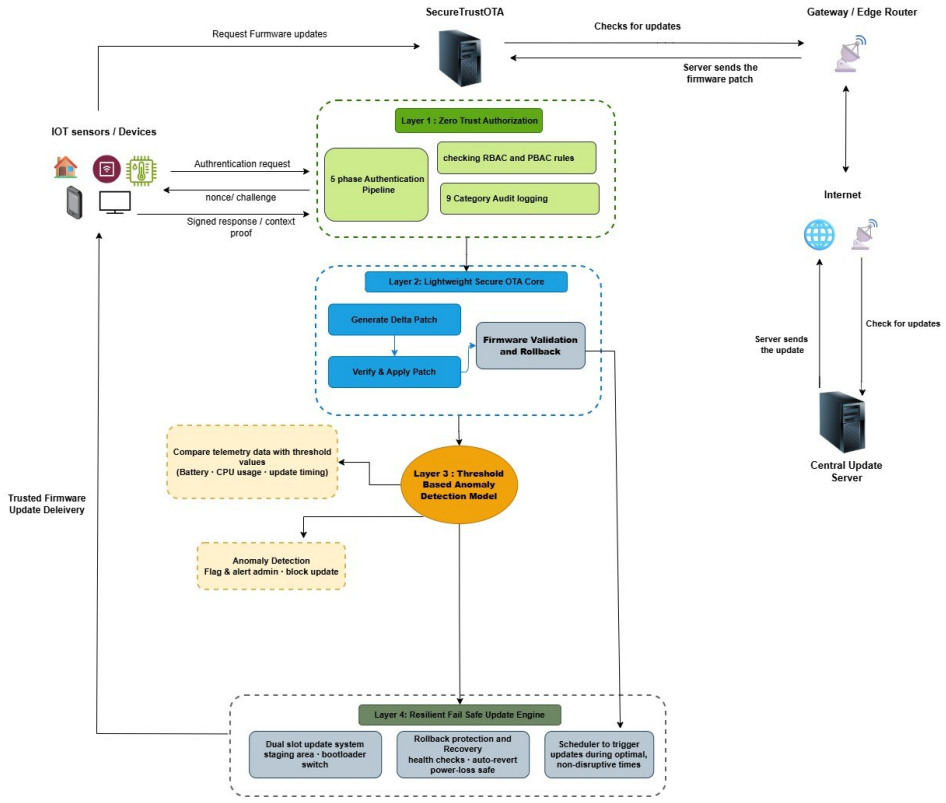

Figure 1: SecureTrustOTA System Architecture

Within this overall architecture, all firmware download requests pass through the Zero Trust Authorization Layer before reaching any downstream delivery service. No update is served without a device first completing the full five phase authentication pipeline and achieving a minimum adaptive trust score of 70/100.

2.2 Zero Trust Authorization Layer - Detailed Design

The Zero Trust Authorization and Policy Enforcement Layer is implemented as a Flask based server that acts as the security gatekeeper for all OTA related device interactions. The system maintains all device state in a SQLite database with nine dedicated audit log tables, and exposes a Role Based Access Control (RBAC) verification endpoint that is called by the downstream firmware delivery server before serving any patch.

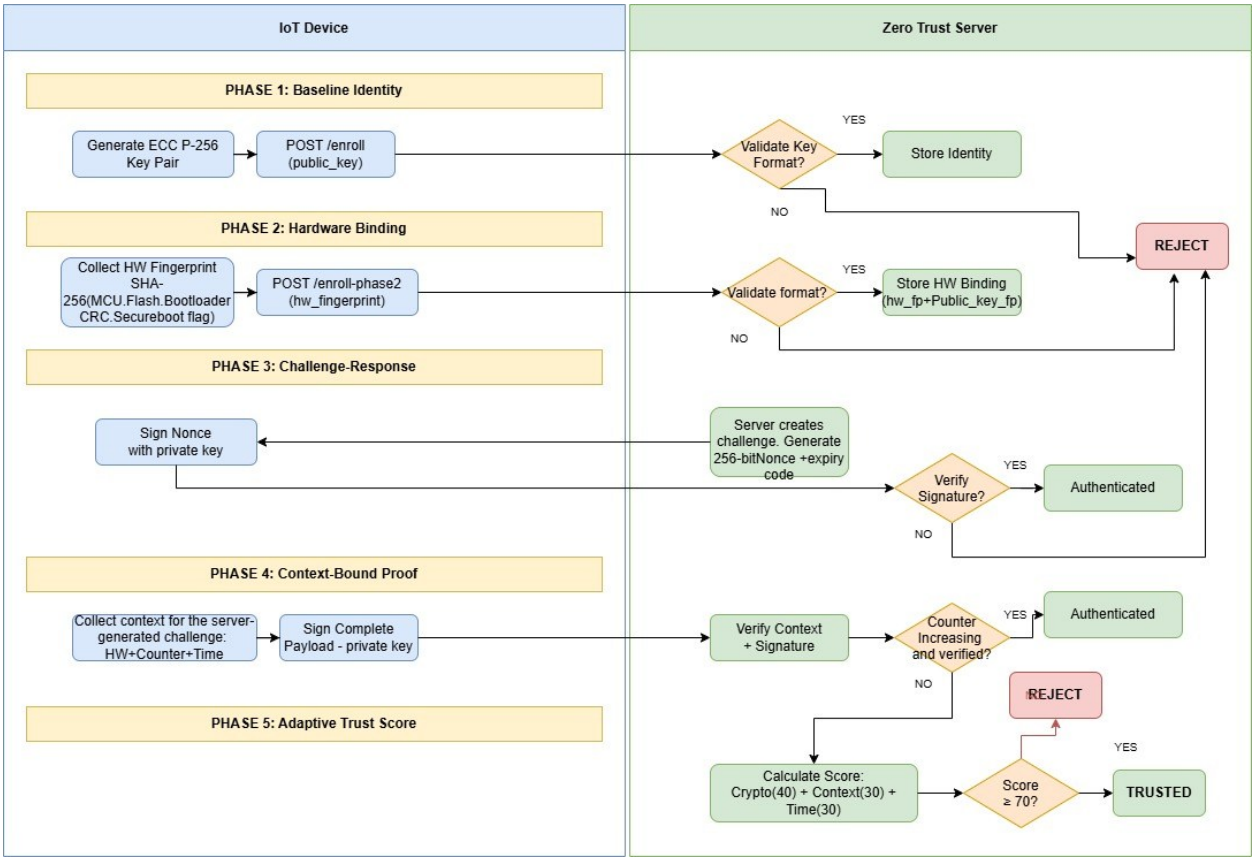

Figure 2: end to end flow of the five phase architecture.

The system operates in two distinct stages. In the enrollment stage, the device registers its cryptographic identity and binds it to its physical hardware characteristics. In the operational stage, every firmware update request triggers the multi-phase authentication sequence that produces a quantitative trust score; only devices exceeding the trust threshold are authorized to receive firmware.

The authentication pipeline follows the principle of progressive trust accumulation: each phase builds on the previous, and failure at any phase terminates the session and logs the event.

2.2.1 Baseline cryptographic identity (phase 1)

In the first phase, the IoT device generates an ECC P-256 key pair locally on the device. Only the public key is transmitted to the server via a POST request to the /enroll endpoint. The server validates the key as a valid PEM-encoded ECC P-256 public key on the SECP256R1 curve, generates a SHA-256 fingerprint of the public key bytes, and stores the device identity record with an active status in the “device_identity” table. The private key never leaves the device, ensuring that compromise of the server does not expose device credentials. This phase establishes a unique, cryptographically verifiable identity for each device.

2.2.2 Hardware bound enrollment (phase 2)

Phase 2 extends the cryptographic identity established in Phase 1 with a binding to the device's physical hardware characteristics. The device collects a hardware fingerprint by hashing a concatenation of hardware specific parameters, MCU unique identifier, flash storage capacity, bootloader CRC, and secure boot flag using SHA-256. A network fingerprint derived from SSID, gateway MAC address, interface type, and VLAN identifier is also collected. Both fingerprints are submitted to the /enroll-phase2 endpoint together with the public key.

The server stores the hardware fingerprint in the “device_hardware_binding” table and associates it with the device's public key fingerprint. On the future enrollments, if the submitted hardware fingerprint does not match the stored binding, the device status is automatically set to quarantine and the mismatch is recorded in the hardware verification log. This mechanism detects physical device replacement, hardware tampering, and cloned device credentials.

2.2.3 Challenge response authentication (phase 3)

Phase 3 establishes proof of private key possession through a standard challenge response protocol. The server generates a cryptographically random 256 bit nonce using a secure random number generator, stores it with a five minute expiry timestamp in the “challenge_nonces” table, and returns it to the device. The device signs the nonce bytes using ECDSA with SHA 256 and returns the signature. The server verifies the signature against the stored public key and marks the nonce as consumed upon successful verification. Single use nonce enforcement prevents replay of captured authentication messages.

2.2.4 Context bound challenge response (phase 4)

Phase 4 requires the device to prove both private key possession and the consistency of its current operational context simultaneously. The server issues a contextual challenge specifying a set of required proof types from the set (hardware state, monotonic counter, timestamp, firmware hash). The device constructs a compound payload by concatenating the challenge nonce with the values of all required context elements, separated by a defined delimiter:

payload = nonce || hw_state_hash || counter || timestamp

This entire compound payload is signed with the device's private key. Signing the full contextual payload rather than only the nonce ensures that the server can verify not just identity but also the device's claimed operational state. The monotonic counter is

incremented and persisted to local storage before the payload is transmitted, ensuring that a process crash between server acceptance and local save cannot leave the counter in a reusable state.

2.2.5 Adaptive trust scoring (phase 5)

Phase 5 implements server side verification of the context bound proof through a three step evaluation that produces a quantitative trust score on a 100-point scale.

Step 5.1: Cryptographic verification (40 points): The server reconstructs the compound payload from the nonce and the claimed context values, then verifies the device's ECDSA signature against its stored public key. This step establishes the cryptographic authenticity of both the identity claim and the context claims simultaneously.

Step 5.2: Context Consistency verification (30 points): The server computes the Hamming distance at the bit level between the hardware fingerprint stored during Phase 2 enrollment and the hardware state hash provided in the current context proof. A 5% Hamming distance drift threshold is applied. values within this bound means minor system variations, while values exceeding it indicate hardware replacement or tampering. Network state drift is also evaluated but treated as informational rather than blocking, since legitimate network environment changes are common.

Step 5.3: Temporal continuity verification (30 points): The server retrieves the last accepted monotonic counter value for the device from the “device_monotonic_counters” table and verifies that the provided counter is strictly greater. Counter reuse triggers a replay attack detection event, a counter value lower than the stored value triggers a rollback attack detection event. Device timestamps, are validated against the server clock with a tolerance of 120 seconds to accommodate legitimate clock skew on constrained devices.

The final trust score is the sum of points awarded across the three steps. Devices achieving a score of 70 or above are classified as “Trusted” and may proceed to OTA

operations. Devices below this threshold are rejected and the full verification report is recorded in the “phase5_verification_log table”.

The trust threshold of 70/100 requires a device to pass cryptographic verification plus at least one of the two contextual checks, providing a degree of operational tolerance while ensuring that no device can be classified as “Trusted” on cryptographic evidence alone.

2.3 Role Based Access Control and Policy Enforcement

The RBAC enforcement endpoint (/rbac/verify) is called by the downstream Delta Patch Server before serving any firmware patch to a device. Critically, the trust score is derived exclusively from the server's own “phase5_verification_log” database. The requesting device cannot supply or influence the score. This design prevents compromised devices from self reporting fraudulent trust scores, a vulnerability present in federated trust systems where trust claims are forwarded between services.

The enforcement logic follows three sequential checks: device existence and enrollment status verification, device status validation (only active devices are permitted; quarantine and revoked devices are blocked), and trust score retrieval from the most recent “Trusted” Phase 5 verification record. Devices that have not completed Phase 4/5 authentication have no “Trsuted” record and are therefore denied access regardless of their enrollment status.

JWT based authorization is implemented using HS256 tokens with device fingerprint binding and role based permissions. IP access management follows a priority based decision logic:

Blocklist check → allowlist check → default policy.

Real time token revocation is supported through a blacklisting mechanism with SQLite persistence, enabling immediate revocation of compromised device credentials without requiring system restart.

2.4 Firmware Attestation and End-to-End Encryption

Before any firmware is delivered to a device, the system performs firmware attestation by computing the SHA-256 hash of the firmware package and comparing it against trusted reference values signed by the manufacturer. Only firmware whose cryptographic hash matches the stored reference and whose digital signature verifies against the trusted public key is approved for delivery. Any mismatch results in rejection of the update with a corresponding log entry.

End to end firmware protection is provided through AES-256 encryption in CBC mode with random initialization vectors (IVs) generated per update transaction. This ensures that even if transmission channels are compromised, the firmware payload cannot be read or tampered with by an intercepting party.

2.5 Audit Logging Architecture

The system maintains nine dedicated audit log tables, each capturing a specific category of security relevant events.

Table 2: Audit log categories

Table	Events Captured
device_action_log	Firmware requests, downloads, decryption, installation
device_auth_log	JWT authentication attempts and outcomes
verification_event_log	Hash checks, signature verifications, attestation results
device_failed_attempt_log	All failed authentication attempts with failure type
system_event_log	Server lifecycle events, severity-classified
hardware_verification_log	Hardware fingerprint binding and drift results
phase5_verification_log	Full trust score breakdowns per verification

challenge_response_log	Phase 3 nonce issuance and verification events
cb_ccr_log	Phase 4/5 context-bound proof events

The “phase5_verification_log” table stores the complete verification report as a JSON document, enabling post incident forensic reconstruction of the full authentication context for any given session. All log entries are timestamped and protected when modified through hash chaining mechanisms.

2.6 Commercialization Aspects

The SecureTrustOTA framework has wide ranging application across multiple sectors where secure and reliable device updates are critical.

Smart Metering and Energy Grids: Utilities can deploy millions of smart meters with confidence, knowing that updates will not disrupt billing or regulatory compliance. The framework ensures continuous operation and minimizes the need for costly technician interventions.

Healthcare IoT Devices: Critical medical monitoring equipment such as wearable devices and bedside IoT sensors requires uninterrupted operation. By providing resilient, trust verified updates, the framework reduces the risk of service failures that could compromise patient safety.

Industrial IoT (IIoT): SCADA systems, manufacturing plants, and other industrial operations demand 24/7 uptime. The lightweight and resilient nature of the OTA framework allows secure updates without interrupting essential production processes.

Smart Homes and Consumer Devices: Everyday IoT devices such as smart plugs, cameras, and appliances benefit from seamless updates that guarantee device safety. Customers can confidently update their devices without the risk of malfunctions.

2.7 Testing and Implementation

2.7.1 Implementation

The implementation was developed as a Flask based Python server with a SQLite backend. The system is organized into the following core components: the enrollment service (Phases 1 and 2), the authentication service (Phases 3, 4, and 5), the RBAC enforcement endpoint, the firmware attestation module, the AES-256 encryption/decryption layer, the JWT management service, and the audit logging subsystem.

The ECC P-256 cryptographic operations are handled using the Python cryptography library. Hardware fingerprinting on the simulated IoT device side is implemented by concatenating MCU UID, flash capacity, bootloader CRC, and secure boot flag strings and computing their SHA-256 digest. The monotonic counter is persisted to a local file with atomic write semantics to prevent counter state loss during unexpected crashes.

2.7.2 Test cases

Subcomponent 1: Role and Policy Based Access Control

Table 3: Test case 001

Test Case	Expected Result
Device with assigned role accessing permitted resource	Authorized actions succeed
Low privilege device attempting admin action	Access denied and event logged
Dynamic policy revocation in real time	Device loses access immediately
Multiple simultaneous policy checks under load	Policies enforced without degradation

Subcomponent 2: Firmware Attestation and Audit Logging

Table 4: Test case 002

Test Case	Expected Result
Firmware integrity attestation during OTA update	Only unmodified firmware passes attestation
Tampered firmware file delivery	Update rejected and event logged
Audit log generation for all access attempts	Every attempt has a timestamped log entry
Log tampering attempt	Logs remain immutable or system detects tampering

Subcomponent 3: Security Attack Scenarios (Experimental Validation)

Table 5: Test case 003

Test Case	Expected Result
Hardware fingerprint tampering (fabricated MCU params)	Device quarantined, HTTP 400 returned
Counter replay attack (reused accepted counter value)	Replay detected, HTTP 401 returned
Forged/invalid signature (deadbeef suffix)	Signature verification fails, HTTP 401 returned
Counter rollback attack (counter below last accepted)	Rollback detected, HTTP 401 returned

CHAPTER 3: RESULTS AND DISCUSSION

3.1 Results

3.1.1 Security validation results

A security failure demonstration was executed against a live deployment of the Zero Trust server. Four attack scenarios were constructed and tested:

Scenario 1: Hardware Fingerprint Tampering: A re-enrollment request was submitted with a hardware fingerprint computed from modified hardware parameters rather than the device's actual MCU characteristics. The server detected the fingerprint mismatch, set the device status to quarantine, and logged the event as “HW_MISMATCH” in the hardware verification log. The enrollment was rejected with HTTP 400.

Scenario 2: Counter Replay Attack: A Phase 4/5 authentication attempt was submitted using a monotonic counter value already accepted by the server in the previous session. The server retrieved the stored counter value, detected that the provided value was not strictly greater, classified the event as a replay attack, and rejected the request with HTTP 401.

Scenario 3: Invalid Signature Forgery: A valid compound payload was constructed and signed correctly, then the last eight hexadecimal characters of the signature were replaced with a fixed value to simulate a forged signature. The server's ECDSA verification failed at Step 5.1 of the Phase 5 pipeline. The request was rejected with HTTP 401.

Scenario 4: Counter Rollback Attack: An authentication attempt was submitted with a monotonic counter value three steps below the last accepted server value. The server detected the rollback condition, logged it as a “ROLLBACK (ATTACK)” event, and rejected the request with HTTP 401.

All four scenarios were correctly detected and rejected. The corresponding events were captured in the “device_failed_attempt_log” and “phase5_verification_log” tables, confirming the completeness of the audit trail. Table 2 summarizes the validation results.

Table 6: Security validation results

Scenario	Attack Type	HTTP Response	Correctly Detected
1	Hardware fingerprint tampered	400	Yes
2	Counter replay attack	401	Yes
3	Invalid/forged signature	401	Yes
4	Counter rollback attack	401	Yes

3.1.2 Performance and latency results

Per phase processing time was measured on the server side under single device conditions across 50 sequential runs. Table 7 summarizes the observed latencies for each authentication phase.

Table 7: Server side phase Latency

Phase	Operation	Average Latency (ms)
1	Public key validation and DB write	392.9
2	Hardware fingerprint binding and DB write	413.92
3	Nonce generation and signature verification	719.68
5	Compound payload construction	785.61
RBAC	Trust score DB lookup and decision	353.51
Total	**End to end authorization**	**2810.34**

3.2 Research Findings

The experimental results confirm that all four major attack vectors - hardware fingerprint tampering, credential replay, signature forgery, and counter rollback are successfully detected and blocked by the proposed five phase authentication pipeline. No false positives were observed for legitimate device authentication in the test environment, confirming that the 5% Hamming distance drift threshold and 120 second clock skew tolerance are appropriately calibrated for the simulated hardware environment.

The end to end authorization latency of approximately 2810 ms, while not negligible, is operationally acceptable for the OTA firmware update use case. Firmware updates are inherently infrequent operations, typically occurring at most several times per week for most IoT deployments, meaning the per transaction overhead has no appreciable impact on device operation. This contrasts with general network authentication where even milliseconds matter. The five phase pipeline intentionally prioritizes security assurance over minimizing per transaction latency, reflecting the asymmetric risk profile of firmware updates. a single unauthorized update can compromise an entire device fleet, whereas an additional authorization overhead is inconsequential for an operation that occurs infrequently.

The trust scoring model's three weighting (40 + 30 + 30) reflects the relative security importance of each dimension. Cryptographic authenticity is necessary but not sufficient; hardware binding and temporal continuity provide the additional assurance that the authenticated device is operating in its expected physical and temporal context. The 70/100 threshold permits a device to pass cryptographic verification plus one contextual check, providing a degree of fault tolerance without permitting a device to be authorized on cryptographic evidence alone.

The server only derivation of trust scores from the “phase5_verification_log” database rather than accepting devic reported scores, closes a critical attack vector present in

federated trust systems. A compromised device is therefore unable to self-report a fraudulent trust score to gain unauthorized access to firmware updates.

3.3 Discussion

The proposed Zero Trust Authorization and Policy Enforcement Layer shows advance features compared to existing OTA security frameworks in several dimensions. Unlike Uptane and TUF, which rely on static enrollment, the proposed system continuously re evaluates device trustworthiness at every transaction through a quantitative multi dimensional trust score. Unlike ASSURED, which uses static authorization tokens, the system enforces per request policy evaluation and incorporates real time hardware state verification. Unlike the Zero Trust MQTT approach of James et al., the system applies Zero Trust principles directly within the OTA firmware authorization process, addressing firmware authenticity, hardware attestation, and comprehensive audit logging, all of which are absent in communication-layer Zero-Trust implementations.

The hardware fingerprinting mechanism addresses a threat category that is entirely absent from existing OTA security frameworks: physical device substitution and hardware level tampering. By binding the ECC P-256 cryptographic identity to the MCU UID, flash capacity, bootloader CRC, and secure boot flag, the system can detect when a device's physical hardware has changed between update cycles, a capability with direct relevance to supply chain security.

The nine-category audit logging architecture provides a level of forensic traceability that exceeds any prior OTA security framework identified in the literature. Every security relevant event from failed authentication attempts and policy enforcement decisions to hardware drift events and trust score breakdowns, is captured in a structured, timestamped format suitable for regulatory compliance and post-incident investigation.

One notable limitation observed during development is the use of SQLite as the backend database. SQLite introduces write contention under high concurrency, which may limit

scalability in deployments involving hundreds of simultaneous device authentication requests. Migration to PostgreSQL with connection pooling is recommended for production deployments at scale. Additionally, the computational overhead on actual constrained hardware (ESP32, STM32) was not formally evaluated in this work. the five-phase pipeline was validated using a software simulator on standard hardware. This represents an important direction for future validation.

CHAPTER 4: CONCLUSION

This dissertation presented a Zero Trust Authorization and Policy Enforcement Layer for IoT OTA firmware delivery, addressing the fundamental limitations of static trust models that characterize existing OTA security frameworks. The system uses a five phase authentication pipeline which combines ECC cryptographic identity, hardware bound enrollment, challenge response authentication, context bound proof, and adaptive trust scoring alongside server side RBAC and a nine category audit logging architecture. By enforcing continuous, hardware based re verification at every update transaction, the system targets vulnerabilities exploited by attacks such as Mirai, detecting and blocking compromised or tampered devices before updates are delivered, thereby limiting their potential impact.

The security validation confirmed resilience against hardware fingerprint tampering, counter replay attacks, signature forgery, and counter rollback attacks, all without requiring specialized trusted hardware extensions such as TPM or ARM TrustZone. The end to end authorization pipeline introduces approximately 2810 ms of latency, which is operationally acceptable for infrequent firmware update operations and represents a justified security investment given the severe consequences of unauthorized firmware installation.

The system's primary contributions to the field are the integration of hardware bound device identity into the OTA authorization decision, the development of a quantitative trust scoring model that enables fine grained authorization decisions, the closure of the self reported trust score attack vector through server only score derivation, and the provision of a comprehensive audit trail that supports regulatory compliance and post-incident forensic analysis.

Future work should focus on evaluating the computational overhead of the five phase pipeline on actual resource constrained hardware such as ESP32 and STM32 microcontrollers, integrating formal certificate revocation infrastructure (OCSP or CRL-

based), adapting the hardware drift detection thresholds for heterogeneous device populations with varying manufacturing tolerances, and migrating the database backend from SQLite to a production-grade relational database to support large-scale concurrent deployments.

REFERENCES

[1] J. L. Hernández-Ramos, M. V. Moreno, J. B. Bernabé, D. Rivera, and A. Skarmeta, "Updating IoT devices: Challenges and potential approaches," in *Proc. Global Internet of Things Summit (GIoTS)*, Dublin, Ireland, Jun. 2020, pp. 1–6.

[2] K. Arakadakis, P. Charalampous, A. Placement, and A. Tryfonas, "Firmware over-the-air programming techniques for IoT networks — a survey," *ACM Comput. Surv.*, vol. 54, no. 9, pp. 1–36, Sep. 2021.

[3] C. Liu, S. Feng, X. Lin, and Z. Yan, "Dissecting zero trust: Research landscape and its implementation in IoT," *Cybersecurity*, vol. 7, no. 1, p. 20, Mar. 2024.

[4] M. James, D. Tracey, and C. Ioannidis, "Authentication and authorization in zero trust IoT: A survey," in *Proc. 35th Irish Signals and Systems Conf. (ISSC)*, Belfast, U.K., Jun. 2024, pp. 1–6.

[5] S. El Jaouhari, "Toward a secure firmware OTA updates for constrained IoT devices," in *Proc. IEEE Int. Smart Cities Conf. (ISC2)*, Paphos, Cyprus, Sep. 2022, pp. 1–7.

[6] N. Asokan, F. Brasser, A. Ibrahim, A.-R. Sadeghi, M. Schunter, G. Tsudik, and C. Wachsmann, "ASSURED: Architecture for secure software update of realistic embedded devices," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 37, no. 11, pp. 2290–2300, Nov. 2018.

[7] S. Hristozov, S. Huber, J. Sigl, and J. Pöppelmann, "Practical runtime attestation for tiny IoT devices," in *Proc. NDSS Workshop on Decentralized IoT Security and Standards (DISS)*, San Diego, CA, USA, Feb. 2018, pp. 1–6.

[8] J. Hallin, "Evaluation of TLS and mTLS in Internet of Things systems," M.S. thesis, Dept. Comput. Sci., Linköping Univ., Linköping, Sweden, 2025.

[9] M. Karimibiuki, K. Fawaz, A. Burg, and P. Kuonen, "Dynpolac: Dynamic policy-based access control for IoT systems," in *Proc. 23rd IEEE Pacific Rim Int. Symp. Dependable Computing (PRDC)*, Taipei, Taiwan, Dec. 2018, pp. 94–103.